QueryBuilder cheat sheet

Builder objektum lekérése, tábla meghatározása

```php
$builder = DB::table("cars");
```

Adatok lekérése

`get()`	Összeállítja és lefuttatja az SQL lekérdezést, majd `stdClass` elemeket tartalmazó `Illuminate\Support\Collection` -t ad vissza.	`$cars = $builder->get();` `echo $cars->first()->model;`
`first()`	Összeállítja és lefuttatja az SQL lekérdezést, majd az eredményt `stdClass` objektumként adja vissza.	`$car = $builder->first();` `echo $car->model;`
`value($column)`	Egyetlen értéked ad vissza. Akkor is, ha több sorból áll. A típusa az adott oszloptól függ.	`$model = $builder->value("model");` `echo $model;`
`pluck($column)`	Összeállítja és lefuttatja az SQL lekérdezést. Az adatbázis megadott oszlopát adja vissza. A visszatérési értéke egy numerikusan indexelt tömbből álló `Illuminate\Support\Collection`.	`$models = $builder->pluck("model");` `echo $models->first();`
`pluck($column,$key)`	Összeállítja és lefuttatja az SQL lekérdezést. Az adatbázis megadott oszlopát adja vissza. A visszatérési értéke egy numerikusan indexelt tömbből álló `Illuminate\Support\Collection`.	`$models = $builder->pluck("model","number_plate");` `echo $models->first();`

Összesítő függvények

`min($column)`	Az adott oszlop **legkisebb** értékét adja meg.	`$minimumConsumption = $builder->min("consumption");` `echo $minimumConsumption;`
`max($column)`	Az adott oszlop **legnagyobb** értékét adja meg.	`$maximumConsumption = $builder->max("consumption");` `echo $maximumConsumption;`
`avg($column)`	Az adott oszlop átlagos értékét adja meg.	`$averageConsumption = $builder->avg("consumption");` `echo $averageConsumption;`
`sum($column)`	Az adott oszlop **összegét** adja meg.	`$sumConsumption = $builder->sum("consumption");` `echo $sumConsumption;`

Lapozó

Tipikusan a `get()` meghívása helyett a `paginate()` metódus meghívásával lehet lapozható oldalt létrehozni.

`paginate($n)`	Beállítja a limitet, hogy az oldal lapozható legyen. Egy oldalon `$n` adat fog megjelenni.	`$cars = $builder->paginate(10);`

Mivel egyes oldalak igen sok adatot tartalmaznak, így célszerű kisebb oldalakra bontani. Erre ad egy nagyon egyszerű megoldást a laravel Pagination néven:

- https://laravel.com/docs/10.x/pagination

A működéséhez három dolog szükséges:

1. Szükség van egy nézetfájlra, ami a lapozó HTML kódját tartalmazza. A Bootstrap 5 megfelelő komponensét tartalmazó kód a `resources/views/vendor/pagination/bootstrap-5.blade.php` fájlban található meg . Ha ez nem található meg, akkor publikálni kell a laravel által már létrehozott nézetfájlokat: `php artisan vendor:publish --tag=laravel-pagination`.
2. Az adatbázisból `get()` helyett `paginate(10)` metódussal kell lekérni az adatokat, így egy oldalon 10 adat fog megjelenni.
3. A nézetben a kapott gyűjteménynek meg kell hívni a `links()` metódust. Ennek a paramétere a nézetfájl elérési útvonala dot notation szerint megadva.

```blade
{{$items->links('vendor.pagination.bootstrap-5')}}
```

A lapozó (itt BS 5) komponens megjelenítése a nézetben

Debug

Metódus	Visszatérési típus	Leírás
`toSql()`	`string`	Visszaadja a behelyettesítésre váró SQL kódot.
`getBindings()`	`array`	Visszaadja a behelyettesítésre váró adatok tömbjét.
`toRawSql()`	`string`	Visszaadja a futtatásra kész SQL kódot.
`dump()`	`Illuminate\Database\Query\Builder`	Megjeleníti a `dump()` függvény segítségével a `toSql()` és a `getBindings()` metódusok által visszaadott adatokat.

Bizonyos metódusok, mint pl a `pluck()` közvetlen a végrehajtás előtt módosítják a Builder-t, így nem tudjuk megnézni milyen SQL kódot futtatott le végül. De a lefuttatott lekérdezések logolhatóak, így valahogy mégis ki lehet nyerni.

```php
// Engedélyezi a lekérdezések logolását
DB::enableQueryLog();

// Lekéri a lefuttatott lekérdezéseket
$queries = DB::getQueryLog();

// Veszi az utolsó lekérdezés logolt adatait
$lastQueryData = end($queries);

// Kiírja a lefuttatott lekérdezést
echo $lastQueryData["query"];
```

Szűrés

Operátorok

Operátor	Megnevezés	Példa
`=`	Egyenlő	`->where("manufacturer","=","Ford")` (`NULL` esetén is `NULL` értéket ad)
`<=>`	NULL safe egyenlőség	`->where("manufacturer","<=>","Ford")` (`NULL` esetén is hamisat értéket ad)
`<>`	Nem egyenlő	`->where("manufacturer","<>","Kia")`
`!=`	Nem egyenlő	`->where("manufacturer","!=","Audi")`
`<`	Kisebb	`->where("consumption","<",6)`
`<=`	Kisebb vagy egyenlő	`->where("consumption","<=",8)`
`>`	Nagyobb	`->where("consumption",">",9)`
`>=`	Nagyobb vagy egyenlő	`->where("consumption",">=",11)`
`like`	Hasonlóság, a joker karakterek alkalmazhatóak	`->where("manufacturer","like","%a")`
`not like`	NEM hasonlóság, a joker karakterek alkalmazhatóak	`->where("manufacturer","not like","%b%")`
`regexp`	Mintaillesztés reguláris kifejezésekre	`->where("number_plate", "regexp","[A-Z]{3}-[0-9]{3}")`
`not regexp`	Nem illeszkedi a reguláris kifejezésre	`->where("number_plate","not regexp", "[A-Z]{3}-[0-9]{3}")`

Egyszerű feltételek

Csak a **diesel üzemanyagú** autók

```php
$builder = DB::table("cars");
$builder->where("fuel","=","diesel");
$cars = $builder->get();
```

Az összes, **NEM diesel** autó

```php
$builder = DB::table("cars");
$builder->where("fuel","<>","diesel");
$cars = $builder->get();
```

Az `A` betűvel kezdődő **rendszámú** autók

```php
$builder = DB::table("cars");
$builder->where("plate_number","like","A%");
$cars = $builder->get();
```

A l**egalább 7.2 litert fogyasztó** autók

```php
$builder = DB::table("cars");
$builder->where("consumption",">=",7.2);
$cars = $builder->get();
```

ÉS

A **benzin** üzemű, **Ford** gyűrtmányú autók

```php
$builder = DB::table("cars");
$builder->where('fuel', '=', 'benzin');
$builder->where('manufacturer', '=', 'Ford');
$cars = $builder->get();
```

VAGY

A benzin, vagy hibrid üzemű autók:

```php
$builder = DB::table("cars");
$builder->where('fuel', '=', 'benzin');
$builder->orWhere('fuel', '=', 'hibrid');
$cars = $builder->get();
```

Rendezés

- `orderBy(column, $direction='asc')`
- `orderByDesc(column)`

Rendezés **gyártó szerint növekvő** sorrendbe

```php
$builder = DB::table("cars");
$builder->orderBy("manufacturer");
$cars = $builder->get();
```

vagy

```php
$builder = DB::table("cars");
$builder->orderBy("manufacturer", "asc");
```

```php
$cars = $builder->get();
```

Rendezés **fogyasztás szerint csökkenő** sorrendbe

```php
$builder = DB::table("cars");
$builder->orderBy("consumption","desc");
$cars = $builder->get();
```

vagy

```php
$builder = DB::table("cars");
$builder->orderByDesc("consumption");
$cars = $builder->get();
```

Összetett példák

Jelenítse meg a kék színű `A` betűvel kezdődő rendszámú autók rendszámát, típusát és gyártóját. Az adatokat gyártó szerint rendezze!

```php
$builder = DB::table("cars");
$builder->where("color", "=","kék");
$builder->where("plate_number", "like", "A%");
$builder->select("plate_number", "manufacturer", "model");
$builder->orderBy("manufacturer");
$cars = $builder->get();
```

rövidebben:

```php
$builder = DB::table("cars")
        ->where("color", "kék")
        ->where("plate_number", "like", "A%")
        ->select("plate_number", "manufacturer", "model")
        ->orderBy("manufacturer");
$cars = $builder->get();
```

még rövidebben:

```php
$cars = DB::table("cars")
        ->where("color", "kék")
        ->where("plate_number", "like", "A%")
        ->select("plate_number", "manufacturer", "model")
        ->orderBy("manufacturer")
        ->get();
```

Mennyit fogyaszt a legkevesebbet fogyasztó jármű? (rendezéssel)

```php
$builder = DB::table("cars");
$builder->orderBy("consumption","asc");
$minimumConsumption = $builder->value("consumption");

echo $minimumConsumption;
```

Mennyit fogyaszt a legkevesebbet fogyasztó jármű? (összesítő függvény alkalmazásával)

```php
$builder = DB::table("cars");
$minimumConsumption = $builder->min("consumption");
```

```
echo $minimumConsumption;
```

Mi a gyártója, típusa és milyen üzemanyaggal megy a legtöbbet fogyasztó autó?

```
$builder = DB::table("cars");
$builder->orderBy("consumption","desc");
$builder->select("manufacturer", "model", "fuel");
$data = $builder->first();

echo "{$data->manufacturer} {$data->model} ({$data->fuel})";
```

Hatékonyság

A Collection és a Builder is rendelkezik összesítő függvényekkel.

A már lekért és Collection-ként visszakapott adatokra kiadni az összesítést egyáltalán nem hatékony megoldás

```
DB::enableQueryLog();

$builder = DB::table("cars");
$minimumConsumption = $builder->get()->min("consumption");
echo $minimumConsumption;

$lastQuery = end(DB::getQueryLog());
echo $lastQuery["query"];
```

A fenti példa esetén az alábbi SQL lekérdezés fut le, azaz **a tábla összes adatát betölti, fölöslegesen**!

```
select * from `cars`
```

Az összesítő függvényeket akkor dolgozza fel az adatbázis, ha azok a Builder objektumra vannak meghívva.

```
DB::enableQueryLog();

$builder = DB::table("cars");
$minimumConsumption = $builder->min("consumption");
echo $minimumConsumption;

$lastQuery = end(DB::getQueryLog());
echo $lastQuery["query"];
```

A fenti példa esetén az alábbi SQL lekérdezést futtatja le az adatbázis motor. Ilyenkor az adatbázis végzi az összesítést, és **egyetlen értéket ad vissza**, a végeredményt

```
select min(`consumption`) as aggregate from `cars`
```